

in this code muzhe bs inta explain kardo ki why $i < \text{high}-1$ that i just comparing till `arr[6]`

```
arr = [5, 3, 8, 4, 2, 7, 1, 10]
```

```
def partition(arr, low, high):  
    pivot = arr[low]  
    i = low, j = high  
    while i < j:  
        while arr[i] <= pivot and i <= high-1:  
            i = i + 1  
        while arr[j] >= pivot and j >= low + 1:  
            j = j - 1  
        if i < j:  
            arr[i], arr[j] = arr[j], arr[i]  
  
    arr[low], arr[j] = arr[j], arr[low]  
    return j
```

Samajhne ke liye step-by-step simple Hinglish me explain karta hoon 👍

Pehle array dekho

python

```
arr = [5, 3, 8, 4, 2, 7, 1, 10]
```

 Copy code

Indexing:

```
0  1  2  3  4  5  6  7
```

 Copy code

`high = 7` (last index)

Tumhara main doubt

👉 `i <= high-1` kyun likha hai?

👉 Kya hum sirf `arr[6]` tak compare kar rahe hain?

Short answer:

- ✓ Haan, `i` sirf `arr[6]` tak jaata hai
- ✓ Taaki `i` **out of bounds** na ho
- ✓ Aur `pivot` se galat comparison na ho

Detail me samjho 👉

Pivot

python

```
pivot = arr[low] # arr[0] = 5
```

 Copy code

Ye loop dekho:

python

```
while arr[i] <= pivot and i <= high-1:  
    i = i + 1
```

 Copy code

high-1 ka matlab

python

```
high = 7  
high-1 = 6
```

 Copy code

So condition ban gayi:

python

```
i <= 6
```

 Copy code

➡ Iska matlab i sirf index 6 (arr[6] = 1) tak hi jaa sakta hai

arr[7] (last element) ko kyun skip kiya?

Because:

1. j already right side se scan kar raha hai

python

```
j = high # j = 7
```

📄 Copy code

2. Agar i ko bhi high tak allow kar diya:

python

```
arr[i] <= pivot
```

📄 Copy code

Toh:

- i = 7 ho sakta hai
 - Next iteration me i = 8
 - ❌ arr[8] → **IndexError**
-

Important Reason (exam / interview point 🔥)

i <= high-1 ensures:

- ✓ i+1 safe rahe
 - ✓ i kabhi high se aage na jaaye
 - ✓ Infinite loop na bane
 - ✓ Array out-of-range error na aaye
-

Simple logic line

- 👉 i left se scan karta hai
- 👉 j right se scan karta hai
- 👉 Dono ko ek hi element (high) par nahi bhejna

One-line summary 🗨️

`i <= high-1` isliye likha hai taaki `i` last element (`arr[7]`) tak na jaaye, kyunki right side ka kaam `j` already kar raha hai, aur index error se bachna bhi zaroori hai.

Agar chaho to mai **dry run karke** poora pointer movement dikha doon 😊



bhai line by line pure code ka dry run karde ga

Bilkul bhai 👍

pure code ka line-by-line DRY RUN karte hain, ekdum slow & clear way me.

Array + indices sab dikhata hoon taaki confusion zero ho jaaye.

◆ Initial Code

python

```
arr = [5, 3, 8, 4, 2, 7, 1, 10]
```

```
def partition(arr, low, high):  
    pivot = arr[low]  
    i = low  
    j = high
```

Values:

Copy code

ini

```
arr    = [5, 3, 8, 4, 2, 7, 1, 10]  
low    = 0  
high   = 7  
pivot  = arr[0] = 5
```

```
i = 0
```

```
j = 7
```

 Copy code

Main while loop

python

```
while i < j:
```

 Copy code

 0 < 7  → enter loop

First inner while (i pointer)

python

```
while arr[i] <= pivot and i <= high-1:  
    i += 1
```

 Copy code

Step-by-step:

- i=0 → arr[0]=5 <= 5  → i=1
- i=1 → arr[1]=3 <= 5  → i=2
- i=2 → arr[2]=8 <= 5  STOP

 i = 2

Second inner while (j pointer)

python

```
while arr[j] >= pivot and j >= low+1:  
    j -= 1
```

 Copy code

Step-by-step:

- j=7 → arr[7]=10 >= 5  → j=6
- j=6 → arr[6]=1 >= 5  STOP

➔ j = 6

Swap check

python

```
if i < j:
    arr[i], arr[j] = arr[j], arr[i]
```

- 2 < 6  → swap

 Copy code

Swap:

```
arr[2] = 8
arr[6] = 1
```

 Copy code

After swap:

ini

```
arr = [5, 3, 1, 4, 2, 7, 8, 10]
```

 Copy code

Loop again

python

```
while i < j:
```

 Copy code

👉 2 < 6 

i pointer again

- i=2 → arr[2]=1 <= 5  → i=3
- i=3 → arr[3]=4 <= 5  → i=4
- i=4 → arr[4]=2 <= 5  → i=5

- $i=5 \rightarrow \text{arr}[5]=7 \leq 5$ ❌ STOP

➡ $i = 5$

j pointer again

- $j=6 \rightarrow \text{arr}[6]=8 \geq 5$ ✅ $\rightarrow j=5$
- $j=5 \rightarrow \text{arr}[5]=7 \geq 5$ ✅ $\rightarrow j=4$
- $j=4 \rightarrow \text{arr}[4]=2 \geq 5$ ❌ STOP

➡ $j = 4$

❌ Swap?

python

```
if i < j:
```

 Copy code

- $5 < 4$ ❌ NO swap
-

Exit loop

python

```
while i < j:
```

 Copy code

- $5 < 4$ ❌ exit
-

Final pivot swap

python

```
arr[low], arr[j] = arr[j], arr[low]
```

 Copy code

Swap:

```
arr[0] = 5  
arr[4] = 2
```

 Copy code

After swap:

ini

```
arr = [2, 3, 1, 4, 5, 7, 8, 10]
```

 Copy code

Return

python

```
return j
```

 Copy code

 j = 4

Final Output

sql

```
Partition index = 4  
Array after partition:  
[2, 3, 1, 4, 5, 7, 8, 10]
```

 Copy code

Meaning:

- Left side of pivot (5) → **smaller elements**
- Right side → **greater elements**
- Pivot at correct position 

One-line summary

i left se chhote elements dhundhta hai, j right se bade, jab galat jagah milte hain toh swap, aur end me pivot apni correct position par aa jaata hai.

Agar chaho:

- 📌 animation style pointer diagram
 - 📌 ya Lomuto vs Hoare comparison
- bas bol bhai 😊



yr i got the logic but how will i use this now in quick sort
and whta the use of returning j

Great question bhai 🧐

Ab tum **real Quick Sort** samajhne ke bilkul edge par ho.

Main simple language + exact connection dikhaata hoon.

1 partition() ka real kaam kya hai?

Partition **poora array sort nahi karta**.

👉 Ye sirf 3 cheezein guarantee karta hai:

nginx

```
low ..... j-1 | j | j+1 ..... high
    < pivot      pivot      > pivot
```

- pivot apni **final correct position** par hota hai
- Left side → pivot se chhote elements
- Right side → pivot se bade elements

Copy code

⚠️ Left & right abhi sorted nahi hote

2 return j kyun kar rahe hain?

python

```
return j
```

 Copy code

👉 j = pivot ka final index

Ye Quick Sort ko batata hai:

“Bhai, pivot yahan settle ho gaya hai, ab iske left aur right ko alag-alag sort kar.”

Without j :

❌ Quick sort ko pata hi nahi chalega

❌ pivot kahan baitha hai

3 Ab Quick Sort ka actual use 🔥

Quick Sort ka logic:

1. Partition karo
2. Pivot index lo (j)
3. Left part sort karo
4. Right part sort karo

4 Full Quick Sort code (tumhare partition ke saath)

python

 Copy code

```
def quickSort(arr, low, high):  
    if low < high:  
        p = partition(arr, low, high) # pivot index  
  
        quickSort(arr, low, p-1) # left side  
        quickSort(arr, p+1, high) # right side
```

5 Dry run (big picture)

Initial:

csharp

 Copy code

```
[5, 3, 8, 4, 2, 7, 1, 10]
```

First call:

python

 Copy code

```
quickSort(arr, 0, 7)
```

Partition → returns $j = 4$

Array:

csharp

 Copy code

```
[2, 3, 1, 4, 5, 7, 8, 10]
```

↑
pivot

Recursive calls:

python

 Copy code

```
quickSort(arr, 0, 3)  # left of pivot  
quickSort(arr, 5, 7)  # right of pivot
```

Left side again partitions:

csharp

 Copy code

```
[2, 3, 1, 4]
```

Pivot = 2 → correct position

Eventually:

csharp

 Copy code`[1, 2, 3, 4, 5, 7, 8, 10]`Sorted 

6 Why $p-1$ and $p+1$?

Because:

- p index already **correct**
- Usko dobara touch nahi karna

less

 Copy code`... p-1 | p | p+1 ...`

7 One killer intuition 🧠

Partition = "Pivot ko throne pe bitha do"

Quick Sort = "Ab left aur right kingdom khud sambhaal lenge (recursion)"

8 Interview-ready answer 🎯

Q: Why partition returns an index?

👉 Because it gives the final position of the pivot, which allows quick sort to recursively sort the left and right subarrays without including the pivot again.

 ChatGPT 

Log in

Sign up for free



Agar chaho next:

- 🔥 pure **QuickSort** ka full dry run
- 🔥 recursion tree diagram
- 🔥 time complexity intuition (best / worst / average)

Bas bol bhai 😊

